

Labo 04 — Labo pratique : construire l'image de l'API

Objectif : écrire vous-même le Dockerfile d'une (fausse) API Spring Boot, et provoquer chacun des pièges du cours — contexte, cache, `CMD` / `ENTRYPOINT`, forme shell, `USER` — avec un moteur de build qui n'a pas de daemon.

Prérequis — Labos 01 à 03 terminés.

Fichiers fournis (`files/`) - `Api.java` — une API HTTP de 30 lignes, sans dépendance. Elle expose `/` et `/actuator/health`, lit `APP_MESSAGE`, `APP_PROFILE`, `SERVER_PORT` dans l'environnement, et gère `SIGTERM`. **Vous n'aurez jamais à la modifier** : les labos portent sur les conteneurs, pas sur Java. - `construire-jar.sh` — compile `Api.java` en `api.jar` dans un conteneur jetable, sans installer de JDK sur votre WSL.

Étape 1 — Préparer le projet

```
mkdir -p ~/labo-docker/04 && cd ~/labo-docker/04
cp <chemin-du-labo>/files/Api.java .
cp <chemin-du-labo>/files/construire-jar.sh . && chmod +x construire-jar.sh
./construire-jar.sh
```

Observez le téléchargement de `docker.io/library/eclipse-temurin:21-jdk` (une seule fois, ~490 Mo), puis `api.jar` d'environ 2,4 Ko.

Explication. Vous venez d'utiliser un conteneur comme **outil jetable** : la compilation a eu lieu dans un JDK complet, monté sur votre dossier, et il n'en reste rien. C'est déjà l'idée du multi-stage du labo 05.

→ JAVA

`javac` compile un `.java` en `.class` (du *bytecode*, indépendant de la machine) ; `jar` empaquette les `.class` dans une archive ZIP avec un manifeste qui dit quelle classe lancer. Le **JDK** (*Development Kit*) contient ces outils ; le **JRE** (*Runtime Environment*) ne contient que ce qu'il faut pour *exécuter* — d'où sa taille moitié moindre, et le choix qu'on fera pour l'image finale.

Étape 2 — Un premier Dockerfile

Créez `Dockerfile` :

```
FROM docker.io/library/eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY api.jar /app/api.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/api.jar"]
CMD ["--spring.profiles.active=prod"]
```

```
podman build -t api-labo:1.0 .
podman images --format 'table {{.Repository}}\t{{.Tag}}\t{{.Size}}' | grep -E 'api-
labo|temurin'
```

Observez les étapes `STEP 1/6` à `STEP 6/6`, chacune suivie d'un identifiant `--> ...`, puis `COMMIT api-labo:1.0` et `Successfully tagged localhost/api-labo:1.0`. L'image pèse **209 Mo** — pour un JAR de 2,4 Ko. L'essentiel est le JRE.

```
podman run -d --name api -p 18080:8080 api-labo:1.0
curl -s localhost:18080/ ; echo
curl -s localhost:18080/actuator/health ; echo
podman logs api
podman port api
```

Observez `{"message":"Bonjour depuis l'API","profile":"default"}`, `{"status":"UP"}`, les lignes `Arguments recus : --spring.profiles.active=prod` puis `API demarree sur le port 8080 (profil default)`, et `8080/tcp -> 0.0.0.0:18080`.

Explication. `EXPOSE 8080` n'a rien publié : c'est `-p 18080:8080` qui a créé la redirection. Vérifiez-le : `podman run -d --name api2 api-labo:1.0` puis `curl -m 2 localhost:18080/` échoue... et rappelez-vous qu'en rootless, un `-p 80:8080` serait refusé.

```
podman rm -f -t 0 api api2
```

→ WINDOWS / WSL

Ouvrez `http://localhost:18080/actuator/health` dans votre navigateur Windows : WSL relaie le port. C'est ainsi que vous testerez le front Angular au labo 07.

Étape 3 — Le contexte de construction

```
mkdir -p ../commun && echo "cle-privee" > ../commun/secret.txt
printf 'FROM docker.io/library/alpine\nCOPY ../commun/secret.txt /\n' > Dockerfile.hors-
contexte
podman build -f Dockerfile.hors-contexte -t essai .
```

Observez l'échec : `Error: building at STEP "COPY ../commun/secret.txt /": ... possible escaping context directory error: copier: stat: "/commun/secret.txt": no such file or directory`.

Explication. Buildah a ramené le chemin à l'intérieur du contexte (`/commun/secret.txt` relatif au dossier) et n'y a rien trouvé. Ce n'est pas un problème de droits : le fichier est hors du périmètre.

Mesurez maintenant le coût d'un contexte non filtré :

```
mkdir -p node_modules && dd if=/dev/zero of=node_modules/gros.bin bs=1M count=200 2>/dev/null
printf 'FROM docker.io/library/alpine\nCOPY . /src\n' > Dockerfile.tout
time podman build -q -f Dockerfile.tout -t api-labo:tout .
podman images --format '{{.Repository}}:{{.Tag}} {{.Size}}' | grep tout
```

Observez un build d'environ 1,7 s... et une image de **218 Mo** pour un `alpine` de 8,7 Mo : les 200 Mo de `node_modules` sont dedans.

```
printf 'node_modules\n*.bin\nDockerfile.hors-contexte\n' > .dockerignore
time podman build -q -f Dockerfile.tout -t api-labo:tout2 .
podman images --format '{{.Repository}}:{{.Tag}} {{.Size}}' | grep tout
```

Observez 8.71 MB pour tout2, et un build plus rapide.

Explication. Avec Docker, ces 200 Mo auraient d'abord été **transférés** au daemon (transferring context) ; Podman lit le dossier sur place, le build semble donc « rapide ». Mais l'image, elle, embarque tout ce que le COPY . . touche. .dockerignore agit **avant** toute instruction : sans lui, node_modules partirait dans l'image publiée. Podman accepte aussi le nom .containerignore — même contenu, même effet.

Étape 4 — Le cache, et l'ordre des instructions

Ajoutez une étape « dépendances » simulée. Remplacez votre Dockerfile par :

```
FROM docker.io/library/eclipse-temurin:21-jre-alpine
WORKDIR /app
RUN echo "telechargement des dependances..." && sleep 5
COPY api.jar /app/api.jar
ENTRYPOINT ["java", "-jar", "/app/api.jar"]
```

```
time podman build -t api-labo:2.0 .
time podman build -t api-labo:2.0 .
```

Observez un premier build d'environ 6,4 secondes, puis un second en 0,7 s, avec --> Using cache sous chaque étape.

Modifiez le JAR et reconstruisez :

```
touch Api.java && ./construire-jar.sh >/dev/null
time podman build -t api-labo:2.1 .
```

Observez que l'étape RUN ... sleep 5 affiche encore --> Using cache : seul le COPY et ce qui suit sont rejoués. 0,7 s.

Maintenant inversez l'ordre — le COPY **avant** le RUN :

```
FROM docker.io/library/eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY api.jar /app/api.jar
RUN echo "telechargement des dependances..." && sleep 5
ENTRYPOINT ["java", "-jar", "/app/api.jar"]
```

```
podman build -t api-labo:3.0 .
./construire-jar.sh >/dev/null && time podman build -t api-labo:3.1 .
```

Observez que les 5 secondes sont **payées à chaque fois** : 6,3 s.

Explication. Voilà, en miniature, la différence entre un build Maven de 40 secondes et un de 6 minutes : le `COPY` volatil placé avant l'étape coûteuse invalide tout ce qui suit.

Étape 5 — `CMD` contre `ENTRYPOINT`

Reconstruisez l'image de l'étape 2 (elle a un `ENTRYPOINT` **et** un `CMD`) et observez la combinaison :

```
timeout 5 podman run --rm api-labo:1.0 | head -2
timeout 5 podman run --rm api-labo:1.0 --debug | head -2
```

Observez dans le premier cas Arguments recus : `--spring.profiles.active=prod`, dans le second Arguments recus : `--debug`. L' `ENTRYPOINT` (`java -jar ...`) est resté ; seul le `CMD` a été remplacé.

Comparez avec une image qui n'a qu'un `CMD` :

```
printf 'FROM docker.io/library/eclipse-temurin:21-jre-alpine\nCOPY api.jar /app/api.jar\nCMD ["java","-jar","/app/api.jar"]\n' > D-cmd
podman build -q -f D-cmd -t api-labo:cmd .
podman run --rm api-labo:cmd sh -c 'echo "je remplace tout"'
```

Observez que l'API ne démarre **pas du tout** : avec un `CMD` seul, l'argument remplace la commande entière.

```
podman run --rm --entrypoint sh api-labo:1.0 -c 'echo "shell obtenu malgré ENTRYPOINT"'
```

Observez que `--entrypoint` est la porte de sortie pour déboguer.

Explication. `CMD` = valeur par défaut remplaçable, `ENTRYPOINT` = programme fixe auquel on ajoute des arguments. Le motif d'entreprise est `ENTRYPOINT` + `CMD` pour les arguments par défaut.

Étape 6 — La forme *shell*, ou l'arrêt cassé

C'est l'expérience la plus importante du labo. Construisez la **même** application avec la forme *shell*, sur une base **Ubuntu** (l'image `21-jre` sans suffixe) :

```
printf 'FROM docker.io/library/eclipse-temurin:21-jre\nCOPY api.jar /app/api.jar\nENTRYPOINT java -jar /app/api.jar\n' > D-shell-debian
podman build -q -f D-shell-debian -t api-labo:shell-debian .
podman run -d --name s-deb api-labo:shell-debian
sleep 3
podman exec s-deb ps -o pid,args | head -3
```

Observez :

```
PID COMMAND
1 /bin/sh -c java -jar /app/api.jar
2 java -jar /app/api.jar
```

Le shell est resté PID 1. Arrêtez :

```
time podman stop s-deb
podman inspect --format 'code={{.State.ExitCode}}' s-deb
podman logs s-deb | tail -2
```

Observez l'avertissement `resorting to SIGKILL`, `real 0m10.8s`, `code=137`, et **aucun** message « SIGTERM reçu » : les *shutdown hooks* n'ont pas été exécutés.

Comparez avec la forme `exec` de l'étape 2 :

```
podman run -d --name s-exec api-labo:1.0
sleep 3 ; podman exec s-exec ps -o pid,args | head -3
time podman stop s-exec
podman inspect --format 'code={{.State.ExitCode}}' s-exec
podman logs s-exec | tail -2
```

Observez `1 java -jar /app/api.jar --spring.profiles.active=prod`, un arrêt en **0,14 s**, `code=143`, et les lignes `SIGTERM reçu : arret propre en cours...` puis `API arretee proprement.`

Enfin, la même forme *shell* mais sur base **Alpine** :

```
printf 'FROM docker.io/library/eclipse-temurin:21-jre-alpine\nCOPY api.jar\n/app/api.jar\nENTRYPOINT java -jar /app/api.jar\n' > D-shell-alpine
podman build -q -f D-shell-alpine -t api-labo:shell-alpine .
podman run -d --name s-alp api-labo:shell-alpine ; sleep 3
podman exec s-alp ps -o pid,args | head -3
time podman stop s-alp ; podman inspect --format 'code={{.State.ExitCode}}' s-alp
```

Observez que cette fois Java **est** PID 1 et que l'arrêt est propre (`143`).

Explication. Le shell de busybox (Alpine) se remplace lui-même par la commande quand celle-ci est simple ; le `dash` d'Ubuntu ne le fait pas. **Le même Dockerfile a donc deux comportements selon l'image de base.** C'est exactement le genre de bogue qui fonctionne sur le poste du développeur et casse en production. La forme `exec` supprime la question.

```
podman rm s-deb s-exec s-alp
```

Étape 7 — Le script d'entrée sans `exec`

Le cas le plus fréquent en entreprise :

```
printf '#!/bin/sh\ncho "preparation..." \njava -jar /app/api.jar\n' > entrypoint.sh
chmod +x entrypoint.sh
printf 'FROM docker.io/library/eclipse-temurin:21-jre-alpine\nCOPY api.jar /app/api.jar\nCOPY\nentrypoint.sh /entrypoint.sh\nENTRYPOINT ["/entrypoint.sh"]\n' > D-script
podman build -q -f D-script -t api-labo:script .
podman run -d --name s-script api-labo:script ; sleep 3
podman exec s-script ps -o pid,args | head -4
time podman stop s-script
podman inspect --format 'code={{.State.ExitCode}}' s-script
```

Observez `1 {entrypoint.sh} /bin/sh /entrypoint.sh` et `2 java -jar /app/api.jar`, puis les 10 secondes et le code `137` — **même sur Alpine**.

Corrigez en ajoutant `exec` :

```
printf '#!/bin/sh\necho "preparation..."\\nexec java -jar /app/api.jar\\n' > entrypoint.sh
podman build -q -f D-script -t api-labo:script2 .
podman rm -f -t 0 s-script && podman run -d --name s-script api-labo:script2 ; sleep 3
podman exec s-script ps -o pid,args | head -3
time podman stop s-script ; podman inspect --format 'code={{.State.ExitCode}}' s-script
podman rm s-script
```

Observez qu'il n'y a plus de shell, l'arrêt est instantané et le code est `143`.

Étape 8 — `ARG` n'est pas un coffre-fort

```
printf 'FROM docker.io/library/alpine\\nARG DB_PASSWORD=vide\\nRUN echo "build avec $DB_PASSWORD"
> /trace.txt\\nCMD ["cat","/trace.txt"]\\n' > D-arg
podman build -q -f D-arg --build-arg DB_PASSWORD='Secr3t!' -t api-labo:arg .
podman image inspect --format '{{json .Config.Env}}' api-labo:arg
podman history --no-trunc api-labo:arg --format '{{.CreatedBy}}' | head -3
```

Observez que `Config.Env` ne contient que `PATH` (c'est vrai, `ARG` ne devient pas `ENV`)... mais que `podman history` affiche `[1 DB_PASSWORD=Secr3t! /bin/sh -c echo "build avec $DB_PASSWORD" > /trace.txt]`.

Explication. Le secret est dans l'image, lisible par quiconque la possède. La bonne méthode viendra au labo 08.

Étape 9 — `USER`, son placement, et ce qu'il devient en rootless

```
printf 'FROM docker.io/library/eclipse-temurin:21-jre-alpine\\nUSER 1000:1000\\nWORKDIR /app\\nRUN
mkdir /data\\nCOPY api.jar /app/api.jar\\nENTRYPOINT ["java","-jar","/app/api.jar"]\\n' > D-user-
tot
podman build -f D-user-tot -t api-labo:user-tot . 2>&1 | grep -iE 'permission|error'
```

Observez l'échec : `mkdir: cannot create directory '/data': Permission denied`, puis `Error: building at STEP "RUN mkdir /data": exit status 1`.

Placez `USER` en dernier :

```
printf 'FROM docker.io/library/eclipse-temurin:21-jre-alpine\\nWORKDIR /app\\nRUN mkdir /data &&
chown 1000:1000 /data\\nCOPY --chown=1000:1000 api.jar /app/api.jar\\nUSER 1000:1000\\nENTRYPOINT
["java","-jar","/app/api.jar"]\\n' > D-user-ok
podman build -q -f D-user-ok -t api-labo:user-ok .
podman run --rm --entrypoint id api-labo:user-ok
```

Observez `uid=1000(...) gid=1000(1000) groups=1000(1000)` : l'application ne tourne plus en root dans le conteneur.

Voyez maintenant ce que cela donne **sur l'hôte** :

```
podman run -d --name u api-labo:user-ok ; podman run -d --name r api-labo:1.0 ; sleep 1
podman top u user,huser,pid,hpid,comm
podman top r user,huser,pid,hpid,comm
podman rm -f -t 0 u r
```

Observez :

USER	HUSER	PID	HPID	COMMAND	<- u : USER 1000 dans l'image
1000	100999	1	12929	java	
USER	HUSER	PID	HPID	COMMAND	<- r : root dans l'image
root	1000	1	13037	java	

Explication. `USER` s'applique à tout ce qui suit, build **et** exécution ; on le place juste avant `ENTRYPOINT`, après avoir préparé les fichiers. En rootless, le « root » du conteneur est déjà votre utilisateur (`HUSER 1000`) ; l'UID 1000 du conteneur, lui, est projeté sur `100999`, un UID de la plage réservée de `/etc/subuid` qui n'a *aucun* droit sur votre hôte. `USER` reste donc utile : il enlève à l'application les privilèges root à l'intérieur du conteneur (fichiers de l'image, ports < 1024, `apk add`), et surtout, la même image tournera un jour sous Docker ou Kubernetes, où root est root.

Nettoyage

```
podman rm -f -t 0 api api2 2>/dev/null
podman rmi api-labo:1.0 api-labo:2.0 api-labo:2.1 api-labo:3.0 api-labo:3.1 \
    api-labo:cmd api-labo:shell-debian api-labo:shell-alpine \
    api-labo:script api-labo:script2 api-labo:arg api-labo:user-ok \
    api-labo:tout api-labo:tout2 2>/dev/null
podman images --format '{{.Repository}}:{{.Tag}}' | grep api-labo
podman rmi $(podman images --filter dangling=true -q) 2>/dev/null
rm -rf ~/labo-docker/04/node_modules ~/labo-docker/04/./commun
```

Gardez `~/labo-docker/04/api.jar` et `Api.java` : les labos 05 à 09 les réutilisent. Conservez aussi les images `eclipse-temurin:21-jre-alpine` et `eclipse-temurin:21-jdk`.

Ce que vous devez pouvoir affirmer maintenant

- Le contexte détermine ce qui est copiable ; avec Podman rien n'est transféré, mais `.dockerignore` reste indispensable pour ce qui finit dans l'image.
- Une instruction invalidée invalide toutes les suivantes : l'ordre décide du temps de build.
- `EXPOSE` ne publie rien.
- `ENTRYPOINT` fixe le programme, `CMD` donne des arguments remplaçables, `--entrypoint` permet de déboguer.
- La forme *shell* peut casser l'arrêt propre — et son comportement dépend de l'image de base. Vous l'avez mesuré : 0,14 s / code 143 contre 10 s / code 137.
- Un script d'entrée doit finir par `exec`.
- Un `--build-arg` est visible dans `podman history`.

- `USER` se place juste avant `ENTRYPOINT` — et garde son sens en rootless.